Machine Translated by Google

SCHOOL OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING AND
INFORMATION TECHNOLOGY

ACADEMIC YEAR 2022-2023, SPRING SEMESTER

APPLICATION DEVELOPMENT METHODOLOGIES

(Laboratory)

Exercise 2: Java I/O

Departments: MAE01, MAE02, MAE04, MAE08, MAE09

(Teacher: A. Georgoulas)

Cognitive subjects covered by the exercises:

- Reading data **(Bytes** & **characters)** from **standard input**
- Read-write files **(FileStreams)**
- Exception handling **(Exceptions)**
- **if-else** and **while** structures
- **String** Handling
- Using the **Scanner** class

General Instructions

The exercise is **optional** and its purpose is to **practice** the topics covered in the lab. Submission of the exercise is NOT required.

To develop the programs, you can use a simple **editor** or **NetBeans.** For additional information on how the classes you will use work, you are encouraged to refer to the relevant documentation that you will find on the official Oracle site as well as other online sources.

Machine Translated by Google

SCHOOL OF ENGINEERING
DEPARTMENT OF COMPUTER AND INFORMATION
TECHNOLOGY ENGINEERING

EXERCISE 2.1: Reading score data from Standard Input

Create ***a console program*** that accepts ***student exam data*** in a given exam period as standard input and prints the **average** grade for each student as standard output. The data given for each student is the student's **registration number ,**
followed by pairs of **courses** and **grades** (separated by a space). We do not know in advance how many course-grade pairs will be given for each student. The end of the *sequence of courses* that the student took is indicated when the word **end** (followed by enter) is given instead of the course name . We also do not know in advance how many students (i.e. registration numbers) will be given grades.
The completion of the program and the end of the data is indicated when the string **000000** is given instead of the **registration number .** The course names are **alphanumeric (Strings)** always with a single word and the grades are **real numbers (Floats).** The registration numbers are alphanumeric.

Indicative data that we give in the input:

```
061125[blank]Programming1[blank]6.1[blank]DB1[blank]7.0[blank]Math1[blank]5.5[blank]end[enter]
071234[blank]DB2[blank]5.5[blank]Programming1[blank]4.0[blank]PSD[blank]4.5[blank]Java[blank]7.3[blank]end[enter]
051058[blank]Math2[blank]7.5[blank]PSD[blank]6.3[blank]end[enter]
081092[blank]end[enter]
071234[blank]DB2[blank]5.5[blank]Math1[blank]5.5[blank]Java[blank]5.0[blank]end[enter]
000000[enter]
```

Indicative results on the screen:

```
Student Average 061125:6.4
Average of Student071234:5.8
...
...
Average of Student 071234:7.2
The reading of the grades was completed successfully.

BUILDSUCCESSFUL(totaltime:0seconds)
```

Hints:

- For reading you can use the **next()** and **nextFloat()** methods provided by the **Scanner class.** The default delimiter is whitespace characters (enter, space, tab).

- To compare a **String** with an alphanumeric constant you can use the **equals() method.** For example use **am.equals("000000").**

Machine Translated by Google

EXERCISE 2.2: Copying files

Create a ***console program*** that will copy the contents of a **text file** to a **second one** (i.e. create a copy). The **name of the file** to be copied, as well as the **name of the copy,** must be given.
as **input** from the user to **standard input** as soon as the program starts. After the copying is finished, an appropriate message should be printed to standard output .

Indicative results on the screen:

```
>Give the full path of the file:
  ...myfile.txt
>Give the full path of the copy:
  ...myfile2.txt
>Copying completed successfully.
BUILDSUCCESSFUL(totaltime:0seconds)
```

Hints:

- You can read the user input with a suitable **Stream** (e.g. **InputStreamReader** or **Scanner).** Correspondingly, for reading and writing the contents you can use the corresponding **Character Streams (FileWriter** and **FileReader).** It is also recommended to convert streams to **BufferedStreams** for better efficiency.
- In case you develop the program with **NetBeans,** if the file you want to copy is in the same directory as the project, you do not need to provide the full **path,** just its name.
- In case you develop the program with a simple **editor,** if the file you want to copy is in the same directory as the class, you do not need to provide a full **path,** just its name.
- For the purposes of this exercise, you do not need to check whether the strings provided by the user are **valid pathnames,** but rather “catch” the error in the corresponding **catch blocks.** Therefore, it is recommended to write code that prints appropriate error messages within the **catch blocks,** so that the user receives information if something goes wrong (e.g., the file is not found, etc.).
- However, you can check if the **path** given by the user is an empty string by using the **isEmpty()** method provided by the **String class.**

Machine Translated by Google

EXERCISE 2.3: Reading a text file and measuring statistics

Create a ***console program*** that will read a text file **(text)** and **display** its **contents** on the screen (standard output) . The name of the text file that we will read (full path) will be given as input by the user to the standard input as soon as the program starts. After displaying the contents of the file, the program should display **statistics** regarding:

- the number of specific Greek characters 'A', 'a', 'C', 'ÿ', 'ÿ' and 'ÿ', which were found in the archive.
- the number of odd digits of the decimal number system ('1', '3', '5', '7', '9') found in the file.

For the purposes of testing your program, you should have created appropriate text files with sample data. For example, suppose you have created the following text file at the location **\...\myfile.txt**

```
AlphabeticalList
==

Georgios Georgiou 9
NikitasN.Karanikolas 7
PanagiotisPanagiotou 10
```

Indicative results on the screen:

```
Enter the file path:
\...\myfile.txt

The contents of the file myfile.txt are:
AlphabeticalList
==
GeorgiosGeorgiou          9
NikitasN.Karanikolas 7
PanagiotisPanagiotou 10

Statistics:
AlphaUpper1
AlphaLower10
GamaUpper2
GammaLower5
OmegaUpper0
OmegaLower1
One 1
```

Machine Translated by Google


```
Three 0
Five 0
Seven 1
Nine 1
BUILDSUCCESSFUL(totaltime:0seconds)
```

Hints:

- To read the file, it is recommended to use **CharacterStream (FileReader** or **BufferedReader)** and not **ByteStream.**
- Also, in order to calculate the statistics, it is recommended to read the characters one by one with the **read() method.**
- The same instructions as the previous exercise apply, regarding the pathnames entered by the user.

EXERCISE 2.4: Printing Fibonacci sequence

Create a **Java** program that calculates and prints the **first 50 terms** of the Fibonacci sequence to a **text file .** The terms should be printed to the file **one per line,** according to the following indicative method:

```
F(0)=0
F(1)=1
F(2)=1
F(3)=2
...
```

In the second phase, add a method that will allow reading this file and printing **only the value of each term** to the standard output.